

PowerShell — Scripting & Automatisation

Mémo de révision rapide · Formation DevSecOps Azure · Simplon

PowerShell 7 · Windows & Linux

CMDLETS ESSENTIELLES — CONVENTION VERBE-NOM

Cmdlet PowerShell	Alias	Équivalent Bash	Rôle
Get-ChildItem	ls, dir	ls	Lister les fichiers
Set-Location	cd	cd	Changer de dossier
Get-Location	pwd	pwd	Afficher le dossier courant
New-Item	—	mkdir / touch	Créer un fichier ou dossier
Copy-Item	cp	cp	Copier
Move-Item	mv	mv	Déplacer / renommer
Remove-Item	rm	rm	Supprimer
Get-Content	cat	cat	Lire un fichier
Write-Output	echo	echo	Afficher du texte
Select-String	—	grep	Rechercher un motif dans un fichier
Where-Object	?	—	Filtrer des objets dans le pipeline
Select-Object	select	—	Choisir des propriétés d'objets
Sort-Object	sort	sort	Trier des objets
Measure-Object	—	wc	Compter / sommer / moyenner
ForEach-Object	%	boucle for	Itérer sur chaque objet du pipeline
Out-File	—	> fichier	Écrire dans un fichier
Export-Csv	—	—	Exporter en CSV
Test-Path	—	-f / -d	Tester si un fichier/dossier existe
Get-Command	gcm	which	Trouver une cmdlet ou commande
Get-Help	help	man	Afficher l'aide

VARIABLES & TYPES

```
# Déclaration
$nom    = "NexaCloud"
$port   = 5001
$sactif = $true

# Typage explicite (recommandé en script)
[string]$nom = "NexaCloud"
[int]$port   = 5001
[bool]$debug = $false
```

```
# Tableau
$niveaux = @("INFO", "WARNING", "ERROR")
$niveaux[0]      # → "INFO"
$niveaux.Count   # → 3

# Hashtable (dictionnaire)
$config = @{
    projet = "NexaCloud"
    port   = 5001
}
$config["port"]  # → 5001
$config.port     # → 5001
```

OPÉRATEURS DE COMPARAISON

Opérateur	Sens	Exemple
<code>-eq</code>	égal	<code>\$a -eq \$b</code>
<code>-ne</code>	différent	<code>\$a -ne \$b</code>
<code>-gt</code>	supérieur	<code>\$a -gt 5</code>
<code>-lt</code>	inférieur	<code>\$a -lt 10</code>
<code>-ge</code>	supérieur ou égal	<code>\$a -ge 3</code>
<code>-le</code>	inférieur ou égal	<code>\$a -le 3</code>
<code>-like</code>	joker <code>*</code>	<code>\$s -like "Err*"</code>
<code>-match</code>	regex	<code>\$s -match "\d+"</code>
<code>-and</code>	ET logique	<code>\$a -gt 0 -and \$b -lt 10</code>
<code>-or</code>	OU logique	<code>\$a -eq 1 -or \$b -eq 2</code>
<code>-not</code>	NON logique	<code>-not \$condition</code>

PowerShell n'utilise pas `==`, `!=`, `>` — toujours les opérateurs avec tiret.

LE PIPELINE ORIENTÉ OBJET — LA DIFFÉRENCE CLÉ AVEC BASH

```
# PowerShell passe des OBJETS (pas du texte brut) d'une cmdlet à l'autre
Get-ChildItem | Where-Object { $_.Length -gt 1024 } | Sort-Object Length -Descending

# $_ = l'objet courant dans le pipeline
Get-ChildItem "*.log" | ForEach-Object { Write-Output $_.Name }

# Compter des éléments : encapsuler dans () puis .Count
(Get-Content "server.log").Count      # nombre de lignes
(Select-String "ERROR" "server.log").Count # nombre de lignes ERROR

# Select-String retourne des objets MatchInfo, pas du texte !
Select-String "ERROR" "server.log" | ForEach-Object { $_.Line }      # texte
Select-String "ERROR" "server.log" | ForEach-Object { $_.LineNumber } # n° ligne

# Exporter en CSV propre
Get-ChildItem | Select-Object Name, Length | Export-Csv "data.csv" -NoTypeInformation
```

CONDITIONS

```
# if / elseif / else
if ($nb -gt 10) {
    Write-Output "Grand"
} elseif ($nb -eq 5) {
    Write-Output "Exactement 5"
} else {
    Write-Output "Petit"
}

# switch (équivalent d'un if/elseif chaîné)
switch ($niveau) {
    "ERROR"    { Write-Output "Erreur !" }
    "CRITICAL" { Write-Output "CRITIQUE !" }
    default    { Write-Output "Inconnu" }
}

# Tester l'existence d'un fichier
if (Test-Path "server.log") { Write-Output "Existe" }
if (-not (Test-Path "server.log")) { Write-Error "Introuvable"; exit 1 }
```

BOUCLES

```
# foreach (boucle classique)
foreach ($n in @("INFO","ERROR")) {
    Write-Output $n
}

# ForEach-Object (pipeline)
1..5 | ForEach-Object { Write-Output "Tour $_" }
# 1..5 génère @(1,2,3,4,5)

# for (numérique)
for ($i = 1; $i -le 5; $i++) {
    Write-Output "Tour $i"
}

# while
$c = 0
while ($c -lt 3) {
    Write-Output $c
    $c++ # raccourci pour $c = $c + 1
}
```

FONCTIONS

```
# Déclarer une fonction
function Compter-Niveau {
    param([string]$Niveau, [string]$Fichier)
```

```

    return (Select-String $Niveau $Fichier).Count
}

# Appel avec paramètres nommés
$nb = Compter-Niveau -Niveau "ERROR" -Fichier "server.log"

# Fonction avec couleur (Write-Host)
function Write-Ok { param($m) Write-Host "[OK] $m" -ForegroundColor Green }

# Paramètre avec valeur par défaut
param([string]$LogFile = "server.log", [int]$Seuil = 3)
# Appel : .\script.ps1 -LogFile "autre.log" -Seuil 5

```

SYNTAXES CLÉS À RETENIR

```

# Here-string (texte multi-lignes)
$texte = @"
Projet : $nomProjet
Date   : $(Get-Date -Format 'dd/MM/yyyy')
"@ # ← le "@" doit être EN DÉBUT DE LIGNE

# Sous-expression dans une chaîne
"Lignes : $($contenu.Count)" #
"Lignes : $contenu.Count"    # × affiche .Count comme texte

# Répétition de chaîne
"=" * 40 # → "====..." (40 fois)

# Objet personnalisé
[PSCustomObject]@{ Niveau = "ERROR"; Count = 5 }

# Arrondi mathématique
[math]::Round($nb * 100 / $total, 1)

# Découper une chaîne
"PORT=5001" -split "=", 2 # → @"PORT","5001")

# Supprimer l'affichage d'une commande
New-Item "fichier" | Out-Null

```

LIRE, ÉCRIRE, EXPORTER

```

# Lire un fichier
$lines = Get-Content "server.log"

# Premières / dernières lignes
Get-Content "server.log" | Select-Object -First 5
Get-Content "server.log" | Select-Object -Last 3

# Écrire dans un fichier (écrase)
$rapport | Out-File "rapport.txt" -Encoding UTF8

# Ajouter à un fichier existant

```

```
Add-Content -Path "rapport.txt" -Value "nouvelle ligne"

# CSV propre (sans ligne de type PowerShell)
Get-ChildItem | Select-Object Name, Length | Export-Csv "data.csv" -NoTypeInformation
```

LANCER UN SCRIPT & POLITIQUE D'EXÉCUTION

```
# Lancer un script (session courante)
.\mon-script.ps1

# Lancer dans une session isolée
pwsh -File mon-script.ps1

# Autoriser les scripts sur Windows (1 seule fois)
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Pas de shebang en PowerShell. L'extension `.ps1` suffit.
Sur macOS/Linux, pas de restriction d'exécution.

COULEURS DANS LE TERMINAL

```
Write-Host "OK" -ForegroundColor Green
Write-Host "Warn" -ForegroundColor Yellow
Write-Host "Erreur" -ForegroundColor Red
Write-Host "Info" -ForegroundColor Cyan
```

PIÈGES COURANTS — ERREURS À NE PAS FAIRE

Erreur fréquente

```
# × .Count sur la commande brute
Get-Content "server.log".Count

# Encapsuler dans ()
(Get-Content "server.log").Count
```

Erreur fréquente

```
# × Propriété sans $()
"Total : $contenu.Count"

# Sous-expression obligatoire
"Total : $($contenu.Count)"
```

Erreur fréquente

```
# × "@ pas en début de ligne
$t = @"
    texte
"@ # ← espaces avant : ERREUR

# Collé à gauche
```

```
$t = @"
texte
"@
```

EXPLORER POWERSHELL — COMMANDES D'AIDE

```
# Aide sur une cmdlet
Get-Help Get-ChildItem
Get-Help Get-ChildItem -Examples
```

```
# Chercher par verbe ou motif
Get-Command -Verb "Get"
Get-Command *Item*
```

```
# Voir les alias
Get-Alias ls
```

```
# Inspecter les propriétés d'un objet
Get-Item "server.log" | Select-Object *
```

```
# Version de PowerShell
$PSVersionTable.PSVersion
```

```
# Variables d'environnement
$env:PATH
$env:PORT # après SetEnvironmentVariable
```